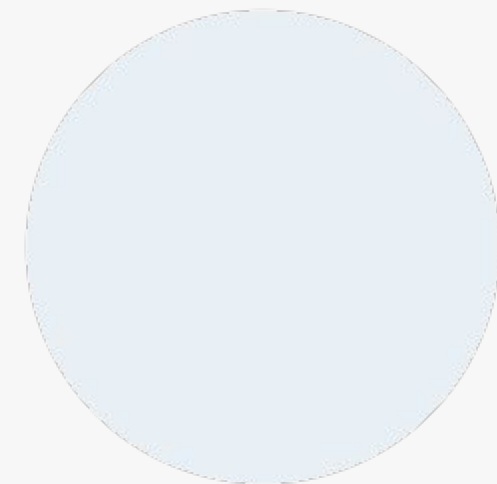


ASTON.



Паттерны



astondevs.ru

Singleton, Multiton



Singleton (Одиночка) — это порождающий паттерн проектирования, который гарантирует, что у класса есть только один экземпляр, и предоставляет к нему глобальную точку доступа.

Одиночка решает сразу две проблемы, нарушая принцип единственной ответственности класса.

- Гарантирует наличие единственного экземпляра класса. Чаще всего это полезно для доступа к какому-то общему ресурсу, например, базе данных.
Представьте, что вы создали объект, а через некоторое время пробуете создать ещё один. В этом случае хотелось бы получить старый объект, вместо создания нового.
Такое поведение невозможно реализовать с помощью обычного конструктора, так как конструктор класса всегда возвращает новый объект.
- Предоставляет глобальную точку доступа. Это не просто глобальная переменная, через которую можно достучаться к определённому объекту. Глобальные переменные не защищены от записи, поэтому любой код может подменять их значения без вашего ведома.

Singleton, Multiton



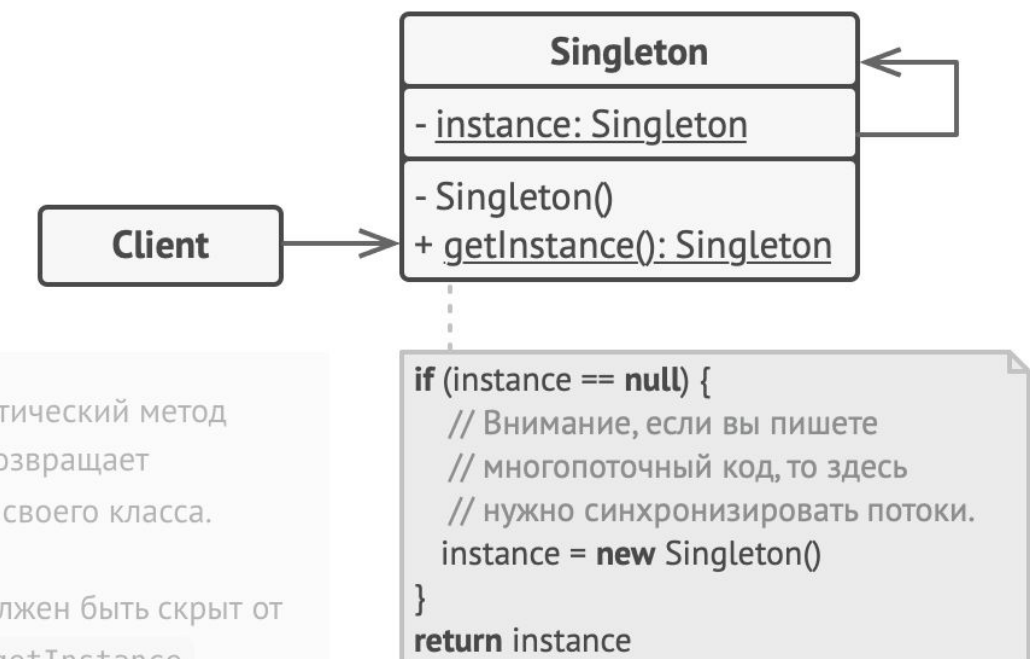
Но есть и другой нюанс. Неплохо бы хранить в одном месте и код, который решает проблему №1, а также иметь к нему простой и доступный интерфейс.

Все реализации одиночки сводятся к тому, чтобы скрыть конструктор по умолчанию и создать публичный статический метод, который и будет контролировать жизненный цикл объекта-одиночки.

Если у вас есть доступ к классу одиночки, значит, будет доступ и к этому статическому методу. Из какой точки кода вы бы его ни вызвали, он всегда будет отдавать один и тот же объект.

1 Одиночка определяет статический метод `getInstance`, который возвращает единственный экземпляр своего класса.

Конструктор одиночки должен быть скрыт от клиентов. Вызов метода `getInstance` должен стать единственным способом получить объект этого класса.



Singleton, Multiton



Ситуации применимости:

Когда в программе должен быть единственный экземпляр какого-то класса, доступный всем клиентам (например, общий доступ к базе данных из разных частей программы).

Одиночка скрывает от клиентов все способы создания нового объекта, кроме специального метода. Этот метод либо создаёт объект, либо отдаёт существующий объект, если он уже был создан.

Когда вам хочется иметь больше контроля над глобальными переменными.

В отличие от глобальных переменных, Одиночка гарантирует, что никакой другой код не заменит созданный экземпляр класса, поэтому вы всегда уверены в наличии лишь одного объекта-одиночки.

Singleton, Multiton



Тем не менее, в любой момент вы можете расширить это ограничение и позволить любое количество объектов-одиночек, поменяв код в одном месте (метод `getInstance`).

Многие программисты считают Одиночку антипаттерном, поэтому его всё реже и реже можно встретить в Java-коде. Тем не менее, Одиночке нашлось применение в стандартных библиотеках Java:

- `java.lang.Runtime#getInstance()`
- `java.awt.Desktop#getDesktop()`
- `java.lang.System#getSecurityManager()`

Признаки применения паттерна: Одиночку можно определить по статическому создающему методу, который возвращает один и тот же объект.

Singleton, Multiton



Шаги реализации:

1. Добавьте в класс приватное статическое поле, которое будет содержать одиночный объект.
2. Объявите статический создающий метод, который будет использоваться для получения одиночки.
3. Добавьте «ленивую инициализацию» (создание объекта при первом вызове метода) в создающий метод одиночки.
4. Сделайте конструктор класса приватным.
5. В клиентском коде замените вызовы конструктора одиночка вызовами его создающего метода.

```
package
refactoring_guru.singleton.example.non_thread_safe;
public final class Singleton {
    private static Singleton instance;
    public String value;
    private Singleton(String value) {
        // Этот код эмулирует медленную инициализацию.
        try {
            Thread.sleep(1000);
        } catch (InterruptedException ex) {
            ex.printStackTrace();
        }
        this.value = value;
    }
    public static Singleton getInstance(String value)
    {
        if (instance == null) {
            instance = new Singleton(value);
        }
        return instance;
    }
}
```


Singleton, Multiton



Шаги реализации:

1. Добавьте в класс приватное статическое поле, которое будет содержать одиночный объект.
2. Объявите статический создающий метод, который будет использоваться для получения одиночки.
3. Добавьте «ленивую инициализацию» (создание объекта при первом вызове метода) в создающий метод одиночки.
4. Сделайте конструктор класса приватным.
5. В клиентском коде замените вызовы конструктора одиночка вызовами его создающего метода.

```
package
refactoring_guru.singleton.example.non_thread_safe;
public class DemoSingleThread {
    public static void main(String[] args) {
        System.out.println("If you see the same value,
then singleton was reused (yay!) " + "\n" +
        "If you see different values, then 2
singletons were created (booo!!) " + "\n\n" +
        "RESULT:" + "\n");

        Singleton singleton =
Singleton.getInstance("FOO");
        Singleton anotherSingleton =
Singleton.getInstance("BAR");

        System.out.println(singleton.value);
        System.out.println(anotherSingleton.value);
    }
}

//If you see the same value, then singleton was reused
(yay!)
//If you see different values, then 2 singletons were
created (booo!)
//RESULT:
//FOO
//FOO
```

Singleton, Multiton



Преимущества и недостатки:

- + Гарантирует наличие единственного экземпляра класса.
- + Предоставляет к нему глобальную точку доступа.
- + Реализует отложенную инициализацию объекта-одиночки.
- Нарушает принцип единственной ответственности класса.
- Маскирует плохой дизайн.
- Проблемы мультипоточности.
- Требуется постоянное создание Mock-объектов при юнит-тестировании.

Singleton, Multiton



Multiton (пул «одиночек») - это по своей сути «реестр» содержащий несколько «одиночек», каждый из которых имеет своё «имя» по которому к нему можно получить доступ.

```
public class Multiton {  
    private static Map instances = new HashMap();  
    private Multiton() {  
        //do setup  
    }  
    public static Multiton getInstance(String key) {  
        Multiton instance = instances.get(key);  
        if(instance==null) {  
            instance = new Multiton();  
            instances.put(key, instance);  
        }  
        return instance;  
    }  
    public void methodThatDoesSomethingInteresting() {  
        //do something interesting  
    }  
}
```

Page Object и Page Factory



Page Object — это всего лишь шаблон проектирования, и многие, кажется, забывают об этом. Это не магия и не что-то космическое; это определенная организация кода, которая создает определенные преимущества.

Page Object - один из наиболее полезных и используемых архитектурных решений в автоматизации. Данный шаблон проектирования помогает инкапсулировать работу с отдельными элементами страницы, что позволяет уменьшить количество кода и упростить его поддержку. Если, к примеру, дизайн одной из страниц изменён, то нам нужно будет переписать только соответствующий класс, описывающий эту страницу.

Основные преимущества:

- Разделение кода тестов и описания страниц
- Объединение всех действий по работе с веб-страницей в одном месте

Page Object и Page Factory



Давайте посмотрим, как с этим работать. Для примера возьмём страницу логина, которая есть практически на всех сайтах. Создадим класс с описание этой страницы, используя шаблон Page Object:

```
public class LoginPage {
    By usernameLocator = By.id("username");
    By passwordLocator = By.id("passwd");
    By loginButtonLocator = By.id("login");
    private final WebDriver driver;
    public LoginPage(WebDriver driver) {
        this.driver = driver;
        if (!"Login".equals(driver.getTitle())) {
            throw new IllegalStateException("This is not the login page");
        }
    }

    public LoginPage typeUsername(String username) {
        driver.findElement(usernameLocator).sendKeys(username);
        return this;
    }

    public LoginPage typePassword(String password) {
        driver.findElement(passwordLocator).sendKeys(password);
        return this;
    }

    public HomePage submitLogin() {
        driver.findElement(loginButtonLocator).submit();
        return new HomePage(driver);
    }

    public LoginPage submitLoginExpectingFailure() {
        driver.findElement(loginButtonLocator).submit();
        return new LoginPage(driver);
    }

    public HomePage loginAs(String username, String password) {
        typeUsername(username);
        typePassword(password);
        return submitLogin();
    }
}
```

Page Object и Page Factory



Объекты страниц обычно соответствуют страницам. Однако большинство современных веб-приложений не строятся как набор уникальных страниц. Вместо этого страницы собираются из набора многократно используемых компонентов. Например, компонент заголовка может находиться в верхней части каждой страницы, а компонент корзины — в правой части большинства страниц, связанных с покупками.

Не имеет смысла дублировать знания о том, как найти элементы в компоненте заголовка, на все объекты страницы, включающие заголовок. Вместо этого создавайте Page Components (компоненты страницы), которые связаны с меньшими частями страниц (в нашем примере — с заголовком).

Еще одним вариантом применения page object шаблона является использование класса Page Factory из библиотеки Selenium. Сначала нам нужно создать простой page object:

```
public class GoogleSearchPage {  
  
    @FindBy (id = "123")  
    private WebElement q;  
    public void searchFor(String text) {  
        q.sendKeys(text);  
        q.submit();  
    }  
}
```

Page Object и Page Factory



Теперь, чтобы всё работало корректно, нам нужно инициализировать наш page object. Это выглядит так:

```
package org.openqa.selenium.example;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.htmlunit.HtmlUnitDriver;
import org.openqa.selenium.support.PageFactory;
public class UsingGoogleSearchPage {
    public static void main(String[] args) {
        WebDriver driver = new HtmlUnitDriver();
        driver.get("http://www.google.com/");
        GoogleSearchPage page =
PageFactory.initElements(driver,
GoogleSearchPage.class);
        page.searchFor("Cheese");
    }
}
```


Page Element



Для построения паттерна Page Element начнем с самого низкоуровневого элемента.

«Виджет» — программный примитив графического интерфейса пользователя, имеющий стандартный внешний вид и выполняющий стандартные действия.

Например, самый простой виджет «Кнопка» — на него можно кликнуть, у него можно проверить текст и цвет.

В «Поле ввода» можно ввести текст, проверить, какой текст введен, кликнуть, проверить отображение фокуса, проверить количество введенных символов, ввести текст и нажать «Enter», проверить placeholder, проверить подсветку «обязательности» поля и текст ошибки, и всё, что еще может понадобиться в конкретном случае.

При этом все действия с этим полем стандартны на любой странице.

InputLine

- InputLine
- InputLine(By)
- clear(): BaseRouter
- fill(String): BaseRouter
- insert(String): BaseRouter
- clearWithFill(String): BaseRouter
- fillWithEnter(String): BaseRouter
- getText(): String
- checkText(String): BaseRouter
- actionsFill(String): BaseRouter
- actionsClear(): BaseRouter
- locator: By

BUTTON

- Button
- Button(By)
- click(): BaseRouter
- hover(): BaseRouter
- getText(): String
- checkText(String): BaseRouter
- waitDisappear(int): BaseRouter
- shouldExist(): BaseRouter
- shouldNotExist(): BaseRouter
- locator: By

Page Element



Бывают виджеты более сложные, для которых действия не так очевидны, например, древовидные оглавления. При их написании нужно отталкиваться от того, что с этой областью программы делает пользователь, например:

- Клик по элементу оглавления с заданным текстом,
- Проверка существования элемента с заданным текстом,
- Проверка отступа элемента с заданным текстом.

Виджеты могут быть двух видов: с локатором в конструкторе и с локатором, вшитым внутрь виджета, без возможности его изменить. Оглавление обычно одно на странице, у него способ поиска на странице можно оставить «внутри» действий с оглавлением, выносить его локатор отдельно нет смысла, так как извне локатор можно случайно испортить, а выгоды от его отдельного хранения нет. В свою очередь текстовое поле — вещь универсальная, с ним наоборот нужно работать только через локатор из конструктора, потому что полей ввода может быть много и сразу. Если появляется хотя бы один метод, предназначенный только для одного специального поля ввода, например с дополнительным кликом по выпадающей подсказке, это уже не просто поле ввода, под него уже пора делать свой виджет.

Page Element



Для уменьшения всеобщего хаоса виджеты, как элементы страницы, объединяются во все те же пейджи, откуда, видимо, и составлено название Page Element.

```
public class UsersPage {  
    public Table usersTable = new Table();  
    public InputLine email = new  
InputLine(By.id("dataEmail"));  
    public InputLine password = new  
InputLine(By.id("dataPassword"));  
    public InputLine name = new  
InputLine(By.id("dataName"));  
    public DropDownList gender = new  
DropDownList(By.id("dataGender"));  
    public Checkbox var11 = new  
Checkbox(By.id("dataSelect11"));  
    public Checkbox var12 = new  
Checkbox(By.id("dataSelect12"));  
    public Checkbox var21 = new  
Checkbox(By.id("dataSelect21"));  
    public Checkbox var22 = new  
Checkbox(By.id("dataSelect22"));  
    public Checkbox var23 = new  
Checkbox(By.id("dataSelect23"));  
    public Button save = new Button(By.id("dataSend"));  
    public ErrorPopup errorPopup = new ErrorPopup();  
    public ModalPopup savePopup = new ModalPopup();  
}
```

Page Element



Для использования всего выше созданного в тестах нужно последовательно обращаться к странице, виджету, действию, таким образом получаем следующую конструкцию:

```
@Test
public void authAsAdmin() {
    baseRouter

    .authPage().email.fill("test@protei.ru")
        .authPage().password.fill("test")
        .authPage().enter.click()

    .mainPage().logoutButton.shouldExist();
}
```

Page Element



Можно добавить классический слой стегов, если есть необходимость для этого в вашем фреймворке (реализация remote библиотеки на Java для RobotFramework требует на вход класс стегов, например), или если хочется добавить аннотаций для красивых отчетов.

```
public class AuthSteps {
    private BaseRouter baseRouter = new BaseRouter ();
    @Step("Sign in as {mail}")
    public BaseSteps login (String mail, String password) {
        baseRouter
            .authPage ().email.fill (mail)
            .authPage ().password.fill (password)
            .authPage ().enter.click ();
        .mainPage ().logoutButton.shouldExist ();
        return this;
    }
    @Step("Fill E-mail")
    public AuthSteps fillEmail (String email) {
        baseRouter .authPage ().email.fill (email);
        return this;
    }
    @Step("Fill password")
    public AuthSteps fillPassword (String password) {
        baseRouter .authPage ().password.fill (password);
        return this;
    }
    @Step("Click enter")
    public AuthSteps clickEnter () {
        baseRouter .authPage ().enter.click ();
        return this;
    }
    @Step("Enter should exist")
    public AuthSteps shouldExistEnter () {
        baseRouter .authPage ().enter.shouldExist ();
        return this;
    }
    @Step("Logout")
    public AuthSteps logout () {
        baseRouter .mainPage ().logoutButton.click ();
        .authPage ().enter.shouldExist ();
        return this;
    }
}

public class BaseRouter {
    // Класс для создания страниц, чтобы не дублировать этот код везде, где понадобится обращение к
    // странице
    public AuthPage authPage () {return page (AuthPage.class);}
    public MainPage mainPage () {return page (MainPage.class);}
    public UsersPage usersPage () {return page (UsersPage.class);}
    public VariantsPage variantsPage () {return page (VariantsPage.class);}
}
```

Page Element



Эти степы очень похожи на степы внутри пейджей, практически ничем не отличаются. Но выделение их в отдельные классы открывает простор для генерации кода, при этом, жесткая связь с соответствующей страницей не теряется.

При этом, если в пейдже не прописывать степы, то смысл инкапсуляции исчезает, а если не добавлять класс степов в `pageElement`, то взаимодействие со страницей всё равно остается отделенным от бизнес-логики.

Таким образом, для составления теста нужно описать простые действия с каждым элементом страницы, причем некоторые элементы вроде полей ввода универсальны и могут быть использованы в нескольких проектах практически без изменений.

Page Element



После этого для каждой страницы указывается набор полей с локаторами. И все, можно писать сам тест, который гарантированно окажется читаемым и воспроизводимым, ведь он состоит из прямых указаний «нажми тут, введи текст там».

Практическое задание



Написать автотесты для сайта wildberries.ru, в ходе которых на главной странице несколько товаров кладется в корзину, затем осуществляется переход в корзину и происходят проверки на соответствие названия товаров, их количество, цен каждого товара в корзине и общей суммы товаров. При написании автотестов необходимо использовать подходящие паттерны, изученные в данной теме.